Table of contents

Questions

1. Describe formally clustering

Intuition:

Clustering is an unsupervised learning technique that groups similar data points together based on their features, without using predefined labels. The goal is to discover natural groupings or patterns in the data.

Detailed Answer:

Formally, given a dataset $X = \{x_1, \dots, x_N\}$ where $x_i \in \mathbb{R}^D$, clustering aims to partition X into K subsets (clusters) $C_1, C_2, \dots, C_K$ such that:

1. $C_k \neq \emptyset$ for all $k = 1, \dots, K$ (non-empty clusters)
2. $\bigcup_{k=1}^{K} C_k = X$ (every point belongs to a cluster)
3. $C_k \cap C_{k'} = \emptyset$ for $k \neq k'$ (clusters are disjoint)

Additionally, we often want points within the same cluster to be more similar to each other than to points in other clusters. The quality of a clustering is typically measured by an objective function, such as minimizing the within-cluster sum of squared distances (in k-means) or maximizing between-cluster separation.

In the context of k-means, clustering is formalized as an optimization problem: find cluster representatives $\mu_1, \dots, \mu_K$ and binary assignment variables z_{ik} that minimize:

$$L(\theta, X) = \sum_{i=1}^{N} \sum_{k=1}^{K} z_{ik} \|x_i - \mu_k\|_2^2$$

subject to $z_{ik} \in \{0, 1\}$ and $\sum_{k=1}^{K} z_{ik} = 1$ for all i.

---

2. In what sense is it an unsupervised technique?

Intuition:

Clustering is unsupervised because it does not use labeled data or target outputs. The algorithm must discover the structure and groupings in the data on its own, without any guidance from predefined categories.

Detailed Answer:

In supervised learning, we have input-output pairs (x_i, y_i) where y_i is a label or target value. The algorithm learns a mapping from inputs to outputs. In contrast, clustering is unsupervised because:

1. **No labels:** We only have input data $X = \{x_1, \ldots, x_N\}$ without any corresponding target values y_i.
2. **Self-discovery:** The algorithm must identify patterns, similarities, and groupings solely from the input data itself.
3. **Objective-driven:** Instead of minimizing prediction error against known labels, clustering algorithms optimize internal criteria like within-cluster similarity or between-cluster separation.
4. **Exploratory nature:** Clustering is often used for exploratory data analysis to discover unknown structures, rather than for prediction.

The k-means algorithm exemplifies unsupervised learning: it partitions data based on proximity to cluster centers, without any external guidance about what clusters should look like or which points belong together.

---

3. Explain why k-means is intrinsically linked to the Euclidean metric?

Intuition:

k-means uses the squared Euclidean distance in its objective function, and the algorithm's update steps rely on properties that are specific to Euclidean geometry, particularly that the mean (centroid) minimizes the sum of squared Euclidean distances.

Detailed Answer:

k-means is intrinsically linked to the Euclidean metric for several reasons:

1. **Objective function:** The loss function is $L = \sum_{i=1}^{N} \sum_{k=1}^{K} z_{ik} \|x_i - \mu_k\|_2^2$, where $\|\cdot\|_2$ is the Euclidean norm. This explicitly uses squared Euclidean distance.
2. **Assignment step:** Each point is assigned to the cluster with the nearest centroid, where "nearest" is measured by Euclidean distance.
3. **Update step:** The cluster centroid μ_k is updated to the mean of points assigned to cluster k: $\mu_k = \frac{1}{N_k} \sum_{i:z_{ik}=1} x_i$. This formula arises because the mean minimizes the sum of squared Euclidean distances:

$$\mu_k = \arg\min_{\mu} \sum_{i:z_{ik}=1} \|x_i - \mu\|_2^2$$

4. **Geometric interpretation:** The algorithm partitions the space into Voronoi cells based on Euclidean distance to cluster centers.

If we used a different metric (e.g., Manhattan distance), the mean would no longer be the minimizer of the sum of distances, and the algorithm would need to be modified (leading to k-medoids or other variants). Thus, k-means is fundamentally designed for Euclidean space.

---

4. Why do we say that k-means considers a Gaussian model for the clusters?

Intuition:

k-means can be derived as a special case of a Gaussian Mixture Model (GMM) where each cluster has a Gaussian distribution with equal, spherical covariance matrices, and where we take the limit as the variance goes to zero.

Detailed Answer:

Consider a Gaussian Mixture Model with K components, where each component k has:

- Weight π_k
- Mean μ_k
- Covariance matrix $\Sigma_k = \sigma^2 I$ (spherical, equal variance)

The EM algorithm for this GMM involves:

- **E-step:** Compute responsibilities $\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i|\mu_k, \sigma^2 I)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i|\mu_j, \sigma^2 I)}$
- **M-step:** Update parameters using weighted statistics

Now consider the limit $\sigma^2 \to 0$:

1. The Gaussian distribution becomes sharply peaked around its mean.
2. The responsibilities γ_{ik} become binary: $\gamma_{ik} = 1$ for the closest μ_k, and 0 otherwise (hard assignment).
3. The M-step update for μ_k becomes the mean of points assigned to cluster k.

This is exactly the k-means algorithm. Therefore, k-means implicitly assumes that each cluster follows a Gaussian distribution with:

- Equal spherical covariances ($\Sigma_k = \sigma^2 I$)
- Equal mixing weights ($\pi_k = 1/K$)
- Infinitesimal variance ($\sigma^2 \to 0$)

Thus, while k-means doesn't explicitly model probabilities, it corresponds to a limiting case of a Gaussian mixture model.

---

5. Is the k-means algorithm exact?

Intuition:

No, k-means is not an exact optimization algorithm. It finds a local optimum rather than the global optimum, and its solution depends heavily on initialization.

Detailed Answer:

k-means is **not exact** in several important ways:

1. **NP-hard problem:** Minimizing the k-means objective function (sum of squared distances) is NP-hard in general. There is no known polynomial-time algorithm that guarantees finding the global optimum.

2. **Local optimum:** The k-means algorithm (coordinate descent) converges to a local minimum of the objective function. Different initializations can lead to different local minima.

3. **Sensitivity to initialization:** The quality of the final clustering depends critically on the initial choice of cluster centers. Poor initialization can lead to suboptimal results.

4. **Approximation guarantees:** While k-means itself has no approximation guarantees, there exist algorithms (like k-means++ initialization) that provide provable approximation bounds for the k-means problem.

5. **Convergence to local minimum:** The algorithm is guaranteed to converge, but only to a local minimum. There is no guarantee that this local minimum is close to the global minimum.

Thus, in practice, k-means is typically run multiple times with different initializations, and the best result (lowest loss) is selected.

---

6. What are the principles to initialize k-means?

Intuition:

Good initialization aims to spread cluster centers across the data space to capture different density regions, which leads to faster convergence and better solutions.

Detailed Answer:

Several initialization principles exist for k-means:

1. **Random initialization:** Randomly select K data points as initial centers. This is simple but can lead to poor results if centers are close together.
2. **k-means++:** A smart initialization method that spreads out the initial centers:
   - Step 1: Choose the first center uniformly at random from the data points.
   - Step 2: For each data point x_i, compute $D(x_i)$, the distance to the nearest already-chosen center.
   - Step 3: Choose the next center from the data points with probability proportional to $D(x_i)^2$.
   - Step 4: Repeat Steps 2-3 until K centers are selected.

   This encourages centers to be far apart and cover different parts of the dataset.
3. **Density-based initialization:** Choose centers in high-density regions of the data.
4. **Farthest-first traversal:** Select centers that are maximally far from each other.
5. **Using domain knowledge:** If prior information about cluster locations is available, use it to seed the centers.

k-means++ is widely used because it provides theoretical guarantees (bounded approximation ratio) and works well in practice.

---

7. What is the coordinate descent algorithm?

Intuition:

Coordinate descent is an optimization algorithm that minimizes a function by iteratively optimizing along one coordinate (variable) at a time, while keeping the other coordinates fixed.

Detailed Answer:

Coordinate descent, also known as alternating minimization, is an iterative optimization technique for minimizing a function $f(x_1, x_2, \dots, x_n)$. The basic idea is:

1. Initialize $x = (x_1, \dots, x_n)$.
2. For $t = 1, 2, \dots$ until convergence:

    - For $i = 1, \dots, n$:
        - Update x_i to minimize $f(x_1, \dots, x_{i-1}, \cdot, x_{i+1}, \dots, x_n)$ while keeping other coordinates fixed.

Mathematically, at iteration t for coordinate i:

$$x_i^{(t+1)} = \arg\min_y f(x_1^{(t+1)}, \dots, x_{i-1}^{(t+1)}, y, x_{i+1}^{(t)}, \dots, x_n^{(t)})$$

Properties:

- Simple to implement if single-variable minimizations are easy.
- May converge slowly if variables are highly correlated.
- Guaranteed to converge for convex, differentiable functions.

Application to k-means:

In k-means, we use block coordinate descent with two blocks: assignments Z and centers M.

- **Minimize over Z (assignment step):** Fix M, assign each point to the nearest center.
- **Minimize over M (update step):** Fix Z, update each center to the mean of its assigned points.

This alternating minimization guarantees that the loss function decreases at each step and converges to a local minimum.
